

Android工程接入

🕒 大约 13 分钟 📄 约 3898 字

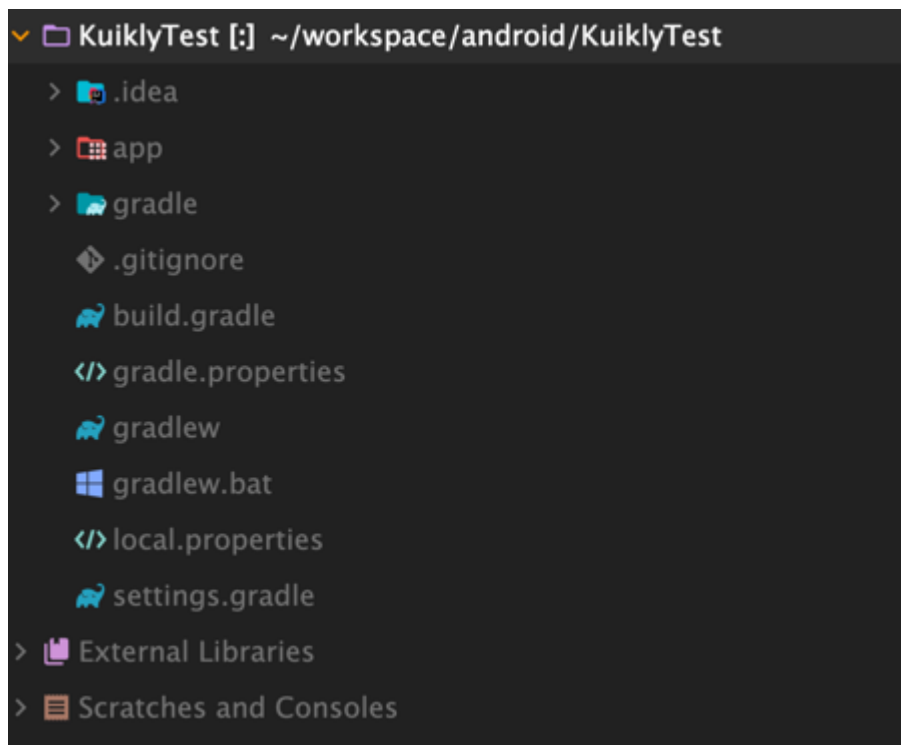
注意

在此之前请确保已经完成**KMP侧 Kuikly**的接入，如还未完成，请移步[KMP跨端工程接入](#)

完成**Kuikly KMP**侧的配置后, 我们还需要将**Kuikly**渲染器和适配器接入到宿主平台中，此文档适用于您想把Kuikly渲染器接入到您现有的Android工程中。下面我们来看下，如何在现有Android工程中接入Kuikly渲染器。

我们先新建一个名为**KuiklyTest**新工程并假设这个工程是你现有的Android工程

反馈文档建议



添加Kuikly渲染器依赖

在引入kuikly的宿主模块（即下方实现Kuikly承载容器的模块）下的gradle文件下添加Kuikly 相关的依赖

注意

此处 core-render-android 和 core 的 Kuikly 版本需要和KMM跨端工程使用的版本保持一致，否则可能会出现兼容性问题

注: 2.5.0版本后需要添加maven源

```
maven("https://mirrors.tencent.com/repository/maven-tencent/
(https://mirrors.tencent.com/repository/maven-tencent/)_")
```

```
dependencies {
    implementation("com.tencent.kuikly-open:core-render-
android:KUIKLY_RENDER_VERSION")
    implementation("com.tencent.kuikly-open:core:KUIKLY_CORE_VERSION")
    implementation 'androidx.core:core-ktx:1.7.0'
    implementation 'androidx.appcompat:appcompat:1.6.1'
    implementation 'com.google.android.material:material:1.8.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.3'
    ...
}
```

gradle

反馈文档建议

提示

- KUIKLY_RENDER_VERSION 需要替换为实际的 kuikly 版本号，在这里[查看最新版本](#)
- KUIKLY_CORE_VERSION 需要替换为实际的 kuikly 版本号，在这里[查看最新版本](#)
- 版本号需要和[KMP跨端工程](#)保持一致

实现Kuikly承载容器

以 Activity 方式接入

在你的android工程新建 KuiklyRenderActivity , 用于承载**Kuikly页面**。具体实现代码，请参考源码工程androidApp模块的 KuiklyRenderActivity 类。

```

class KuiklyRenderActivity : AppCompatActivity() {

    private lateinit var hrContainerView: ViewGroup
    private lateinit var loadingView: View
    private lateinit var errorView: View

    private lateinit var kuiklyRenderViewDelegator: KuiklyRenderViewBaseDelegator

    protected val pageName: String
        get() {
            val pn = intent.getStringExtra(KEY_PAGE_NAME) ?: ""
            return if (pn.isNotEmpty()) {
                return pn
            } else {
                "router"
            }
        }

    private lateinit var contextCodeHandler: ContextCodeHandler

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // 1. 创建一个Kuikly页面打开的封装处理器
        contextCodeHandler = ContextCodeHandler(pageName)
        // 2. 实例化Kuikly委托者类
        kuiklyRenderViewDelegator = contextCodeHandler.initContextHandler()
        setContentView(R.layout.activity_hr)
        setupImmersiveMode()
        // 3. 获取用于承载Kuikly的容器View
        hrContainerView = findViewById(R.id.hr_container)
        loadingView = findViewById(R.id.hr_loading)
        errorView = findViewById(R.id.hr_error)
        // 4. 触发Kuikly View实例化
        // hrContainerView: 承载Kuikly的容器View
        // contextCode: jvm模式下传递""
        // pageName: 传递想要打开的Kuikly侧的Page名字
        // pageData: 传递给Kuikly页面的参数
        contextCodeHandler.openPage(this, hrContainerView, pageName,
createPageData())
    }

    override fun onResume() { // 5. 通知Kuikly页面触发onResume
        super.onResume()
        kuiklyRenderViewDelegator.onResume()
    }
}

```

```
override fun onPause() { // 6. 通知Kuikly页面触发onStop
    super.onPause()
    kuiklyRenderViewDelegator.onPause()
}

override fun onDestroy() { // 7. 通知Kuikly页面触发onDestroy
    super.onDestroy()
    kuiklyRenderViewDelegator.onDetach()
}

private fun createPageData(): Map<String, Any> {
    val param = argsToMap()
    param["appId"] = 1
    return param
}

private fun argsToMap(): MutableMap<String, Any> {
    val jsonStr = intent.getStringExtra(KEY_PAGE_DATA) ?: return
mutableMapOf()
    return JSONObject(jsonStr).toMap()
}

private fun setupImmersiveMode() {
    window?.apply {
addFlags(WindowManager.LayoutParams.FLAG_DRAWS_SYSTEM_BAR_BACKGROUNDS)
        statusBarColor = Color.TRANSPARENT
        decorView?.systemUiVisibility =
            View.SYSTEM_UI_FLAG_LAYOUT_FULLSCREEN or
View.SYSTEM_UI_FLAG_LAYOUT_STABLE
        decorView.systemUiVisibility = View.SYSTEM_UI_FLAG_LAYOUT_FULLSCREEN
or
            View.SYSTEM_UI_FLAG_LAYOUT_STABLE or
View.SYSTEM_UI_FLAG_LIGHT_STATUS_BAR
    }
}

companion object {
    private const val TAG = "KuiklyRenderActivity"
    private const val KEY_PAGE_NAME = "pageName"
    private const val KEY_PAGE_DATA = "pageData"

    fun start(context: Context, pageName: String, pageData: JSONObject) {
        val starter = Intent(context, KuiklyRenderActivity::class.java)
```

```

        starter.putExtra(KEY_PAGE_NAME, pageName)
        starter.putExtra(KEY_PAGE_DATA, pageData.toString())
        context.startActivity(starter)
    }
}
}

```

```

// activity_hr.xml
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <FrameLayout
        android:id="@+id/hr_container"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

    <View
        android:id="@+id/hr_loading"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

    <View
        android:id="@+id/hr_error"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

</FrameLayout>

```

xml

反馈
文档
建议

以 View 方式接入

除了使用 Activity 作为 Kuikly 页面的容器外，Kuikly 还支持以 **View 粒度** 直接嵌入到任意原生视图层级中。这种方式适用于在一个 Native Activity/Fragment 中嵌入一个或多个 Kuikly 子视图（如瀑布流卡片、Banner 等混合场景）。

代码示例

```

class NativeMixKuiklyViewDemoActivity : AppCompatActivity() {

    private var kuiklyView: KuiklyBaseView? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_native_mix_kuikly_view)
        val rootView = findViewById<ViewGroup>(R.id.root_view)
        // 实例化Kuikly委托者类
        val delegate = object : KuiklyRenderViewBaseDelegatorDelegate {
            // any implement.....
        }
        // 创建KuiklyBaseView并附加到容器
        kuiklyView = KuiklyBaseView(this, delegate)
        // 加载Kuikly页面: contextCode传空字符串, pageName为页面名称, pageData为页面
        kuiklyView?.onAttach("", "yourPageName", mapOf())
        rootView.addView(kuiklyView)
    }

    override fun onPause() {
        super.onPause()
        kuiklyView?.onPause()
    }

    override fun onResume() {
        super.onResume()
        kuiklyView?.onResume()
    }

    override fun onDestroy() {
        super.onDestroy()
        kuiklyView?.onDetach()
    }
}

```

参数

反馈
文档
建议

完整的 View 粒度接入实践示例（卡片式风格瀑布流），请参考源码工程 androidApp 模块的 NativeAppWaterfallActivity 类，该 Demo 在一个 RecyclerView 的每个 ViewHolder 中各嵌入了一个 KuiklyBaseView。

与 Activity 方式的主要不同点

	Activity 方式	View 方式
容器	KuiklyRenderViewDelegator	KuiklyBaseView （继承 FrameLayout
入口时机	Activity 的 onCreate 中通过 Delegator 加载	同样在 onCreate 中创建 KuiklyBaseView 并调用 onAttach() 载页面
生命周期	由 Delegator 在 Activity 的 onResume / onPause / onDestroy 中自动管理	需业务手动调用 KuiklyBaseView 的 onResume() 、 onPause() 、 onDetach()
尺寸控制	自动撑满整个 Activity	通过 LayoutParams 自行指定尺寸
代理协议	实现 KuiklyRenderViewBaseDelegatorDelegate	同样实现 KuiklyRenderViewBaseDelegatorDelegate （能力一致）

反馈文档建议

注意事项

- 1. **创建时机**：在 Activity 的 onCreate 中创建 KuiklyBaseView 并调用 onAttach(contextCode, pageName, pageData) 加载 Kuikly 页面
- 2. **生命周期手动转发**：必须在宿主 Activity/Fragment 的 onResume() 中调用 kuiklyView.onResume() 、 onPause() 中调用 kuiklyView.onPause() 、 onDestroy() 中调用 kuiklyView.onDetach()
- 3. **pageData 扁平传递**： pageData 传入扁平的 Map<String, Any> 即可，框架内部会自动将其包裹在 param key 下，**不需要业务侧额外包裹**
- 4. **代理能力一致**：View 方式的 KuiklyRenderViewBaseDelegatorDelegate 与 Activity 方式需要实现的代理完全一致

view 粒度接入进的原生页面，还要在 KRRouterAdapter 中添加 pageName 路由分支，并在 AndroidManifest.xml 中注册新建的 Native Activity。

实现Kuikly适配器（必须实现部分）

Kuikly 框架为了灵活和可拓展性，不会内置实现图片下载，异常处理，日志实现等功能，而是通过适配器的设计模式，将具体实现委托给宿主App实现。

Kuikly 目前含有以下适配器, 需宿主平台按需实现

1. **图片加载适配器**: 用于给Kuikly的Image组件实现图片下载解码能力。**宿主侧必须实现**
2. **日志适配器**: 用于给Kuikly框架和Kuikly业务实现日志打印。**宿主侧必须实现**
3. **页面路由适配器**: 用于实现跳转到 Kuikly 容器。**宿主侧必须实现**
4. **线程适配器**: Kuikly 内部不会创建子线程, 复用宿主的子线程, 防止占用宿主太多资源。

宿主侧必须实现

5. **异常适配器**: 当Kuikly业务执行逻辑出错时, 决定如何处理异常。**推荐宿主侧实现**
6. **颜色值转换适配器**: Kuikly框架对颜色值的处理, 默认只处理十六进制的颜色值。**宿主按需实现**
7. **自定义字体适配器**: Kuikly框架不会内置一些自定义字体。业务如果有自定义字体的需求, 需实现此适配器来扩展字体。**宿主按需实现**
8. **APNG图片加载适配器**: 用于给Kuikly提供APNG图片加载的能力。**宿主按需实现**（使用APNG组件时必须实现, 可参考 [KRAPNGViewAdapter.kt \(https://github.com/Tencent-TDS/KuiklyUI/blob/main/androidApp/src/main/java/com/tencent/kuikly/android/demo/adapters/KRAPNGViewAdapter.kt\)](https://github.com/Tencent-TDS/KuiklyUI/blob/main/androidApp/src/main/java/com/tencent/kuikly/android/demo/adapters/KRAPNGViewAdapter.kt)）
9. **PAG加载适配器**: 用于给Kuikly提供PAG加载的能力。**宿主按需实现**（使用PAG组件时必须实现, 可参考 [PAGViewAdapter.kt \(https://github.com/Tencent-TDS/KuiklyUI/blob/main/androidApp/src/main/java/com/tencent/kuikly/android/demo/adapters/PAGViewAdapter.kt\)](https://github.com/Tencent-TDS/KuiklyUI/blob/main/androidApp/src/main/java/com/tencent/kuikly/android/demo/adapters/PAGViewAdapter.kt)）

实现图片适配器

具体实现代码, 请参考源码工程androidApp模块的 KRIImageAdapter 类。

```
object KRImageAdapter : IKRImageAdapter {
    override fun fetchDrawable(
        imageLoadOption: HRIImageLoadOption,
        callback: (drawable: Drawable?) -> Unit
    ) {
        TODO("Not yet implemented")
    }
}
```

kt

完成后，可通过**模版工程**中的 ImageAdapter基准测试 页面来验证功能正常，可能需要重载 IKRImageAdapter 的 getDrawableWidth 和 getDrawableHeight 方法调节渲染效果。

注意

fetchDrawable 方法可能在非UI线程调用，例如在 MemoryCacheModule.cacheImage 中（可参考示例(<https://github.com/Tencent-TDS/KuiklyUI/blob/main/demo/src/commonMain/kotlin/com/tencent/kuikly/demo/pages/demo/CanvasTestPage.kt>)）。实现时需要注意线程安全，如果需要在UI线程操作（如更新UI组件），请使用 Handler 或 runOnUiThread 等方式切换到UI线程。

反馈文档建议

注意

框架默认会把图片在首屏完成后在进行加载(优化首屏性能)，若不希望框架对此做异步可以在 KRImageAdapter 重写 shouldWaitViewDidLoad 字段为 false 以实现

实现日志适配器

具体实现代码，请参考源码工程androidApp模块的 KRLogAdapter 类。

```
object KRLogAdapter : IKRLogAdapter {
    override val asyncLogEnable: Boolean
        get() = TODO("Not yet implemented")

    override fun i(tag: String, msg: String) {
        TODO("Not yet implemented")
    }
}
```

kt

```

        override fun d(tag: String, msg: String) {
            TODO("Not yet implemented")
        }

        override fun e(tag: String, msg: String) {
            TODO("Not yet implemented")
        }
    }
}

```

实现异常适配器

具体实现代码，请参考源码工程androidApp模块的 `KRUNcaughtExceptionHandlerAdapter` 类。

反馈文档建议

```

object KRExceptionAdapter : IKRUNcaughtExceptionHandlerAdapter {
    override fun uncaughtException(throwable: Throwable) {
        if (BuildConfig.DEBUG) { // debug版本crash
            throw throwable
        } else {
            Log.d("KRError", throwable.stackTraceToString()) // release版本打印日志
        }
    }
}

```

kt

页面路由适配器

该适配器必须实现, 用于实现 Kuikly 页面之间的跳转

具体实现代码，请参考源码工程androidApp模块的 `KRRouterAdapter` 类。

```

object KRRouterAdapter : IKRRouterAdapter {

    override fun openPage(
        context: Context,
        pageName: String,
        pageData: JSONObject,
    ) {

```

kt

```

        KuiklyRenderActivity.start(context, pageName, pageData)
    }

    override fun closePage(context: Context) {
        (context as? Activity)?.finish()
    }
}

```

线程适配器

该适配器必须实现，用于让 Kuikly 的任务在子线程执行

具体实现代码，请参考源码工程androidApp模块的 `KRThreadAdapter` 类。

```

class KRThreadAdapter : IKRThreadAdapter {
    override fun executeOnSubThread(task: () -> Unit) {
        execOnSubThread(task)
    }

    private val subThreadPoolExecutor by lazy {
        Executors.newFixedThreadPool(2)
    }

    fun execOnSubThread(runnable: () -> Unit) {
        subThreadPoolExecutor.execute(runnable)
    }
}

```

kt

反馈
文档
建议

stackSize（线程栈大小配置）

`IKRThreadAdapter` 接口还提供了 `stackSize()` 方法，用于配置 Kuikly 内部线程的栈大小，主要用于在 Compose 场景下避免 布局嵌套过深导致的 `StackOverflowException`

实现示例：

基础实现（使用系统默认值）：

```
class KRThreadAdapter : IKRThreadAdapter {
    // 使用系统默认线程大小（通常为1MB）（返回 0）
    override fun stackSize(): Long = 0
}
```

kt

自定义栈大小（推荐用于 Compose 场景）：

```
class KRThreadAdapter : IKRThreadAdapter {
    /**
     * 在 Compose 场景下，建议使用 8MB 或更大的栈大小以避免 StackOverflowException
     */
    override fun stackSize(): Long {
        return 8 * 1024 * 1024 // 8MB
    }
}
```

kt

使用建议：

- 大多数场景下，系统默认值（1MB）已经足够
- 如果遇到 StackOverflowException，可以尝试设置为 8 * 1024 * 1024（8MB）
- 在 Compose 场景下，建议使用 8MB 或更大的栈大小
- 注意：过大的栈大小会占用更多内存，建议根据实际需求设置

其他按需实现适配器示例参考[实现适配器（按需实现部分）](#)

将适配器设置给Kuikly

具体实现代码，请参考源码工程androidApp模块的 KuiklyRenderActivity 类。

```
class KuiklyRenderActivity : AppCompatActivity(),
    KuiklyRenderViewDelegatorDelegate {
    ...
    companion object {
        init {
            // 初始化Kuikly适配器
            initKuiklyAdapter()
        }

        private fun initKuiklyAdapter() {
            with(KuiklyRenderAdapterManager) {

```

kt

```

        krImageAdapter = KRIImageAdapter
        krLogAdapter = KRLogAdapter
        krUncaughtExceptionHandlerAdapter =
KRUncaughtExceptionHandlerAdapter
        krRouterAdapter = KRRouterAdapter
        krThreadAdapter = KRThreadAdapter()
        // 按需实现其他适配器
        // krFontAdapter = KRFontAdapter
        // krColorParseAdapter =
KRColorParserAdapter(KRApplication.application)
    }
}
...
}

```

编写TestPage验证

来到这步后, 我们已经完成Kuikly平台侧的接入, 下面我们在[KMP跨端工程接入](#)创建的KMP工程中的 shared 模块下新建 TestPage , 测试我们的接入是否成功

```

@Page("test")
class TestPage : Pager(){
    override fun body(): ViewBuilder {
        return {
            attr {
                allCenter()
                backgroundColor(Color.WHITE)
            }

            Text {
                attr {
                    fontSize(20f)
                    color(Color.GREEN)
                    text("Hello Kuikly")
                }
            }
        }
    }
}

```

kt

将业务代码集成到现有工程

1. 编写完业务代码后, 我们需要将业务代码编译成aar集成到现有工程, 集成方式可以是**远程依赖**或者**本地依赖**的方式。
2. 我们在**KMP业务工程**中执行 `./gradlew :shared:bundleDebugAar` 命令, 将业务代码编译成AAR, AAR产物位于 `shared/build/output/aar` 中
3. 将产物集成到你现有的工程中

跳转到KuiklyRenderActivity容器

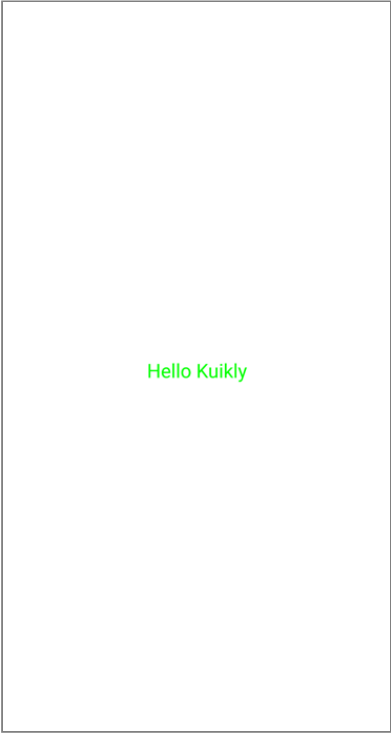
1. 在合适的时机跳转到 `KuiklyRenderActivity` 容器, 并指定跳转的url为 `TestPage` 对应的 `pageName: test`

```
kuiklyRenderActivity.start(context, "test", JSONObject())
```

kt

反馈
文档
建议

在AndroidApp, 当出现下面界面时, 证明已经接入成功



Hello Kuikly

实现适配器（按需实现部分）

实现颜色值转换适配器

通过该适配器自定义颜色转换逻辑，业务可根据实际使用需求来决定是否实现
具体实现代码，请参考源码工程androidApp模块的 `KRColorParserAdapter` 类。

```
object KRColorAdapter : IKRColorParserAdapter {  
    override fun toColor(colorStr: String): Int? {  
        // 自定义转换颜色  
    }  
}
```

kt

定义字体适配器

该适配器扩展自定义字体，以及控制系统显示大小缩放行为。

加载自定义字体

从 assets 加载自定义字体文件：

```
object KRFontAdapter : IKRFontAdapter {  
    override fun getTypeface(fontFamily: String, result: (Typeface?) -> Unit) {  
        // 加载自定义字体，结果通过 result 回调给 Kuikly 侧  
  
        // 例: assets/fonts/ 存放 Satisfy-Regular.ttf 自定义字体文件  
        if (fontFamily.isEmpty()) {  
            result(null)  
        } else {  
            var tfe: Typeface? = null  
            when (fontFamily) {  
                "Satisfy-Regular" -> {  
                    tfe =  
Typeface.createFromAsset(KRApplication.application.assets,  
"fonts/$fontFamily.ttf")  
                }  
            }  
            result(tfe)  
        }  
    }  
}
```

kt

```

        }
    }
    result(tfe)
}
}
}

```

实现不跟随系统显示大小变化

通过固定 density 值，使 Kuikly 页面布局不受系统"显示大小"设置影响。

步骤 1：在 FontAdapter 中实现固定 density

```

object KRFontAdapter : IKRFontAdapter {
    override fun getDisplayMetrics(useHostDisplayMetrics: Boolean?):
    DisplayMetrics {
        return DisplayMetrics().apply {
            density = 2f
            scaledDensity = 2f
        }
    }
}

```

kt

反馈
文档
建议

步骤 2：在 Delegate 中启用 useHostDisplayMetrics

```

val delegate = object : KuiklyRenderViewBaseDelegatorDelegate {
    override fun useHostDisplayMetrics(): Boolean = true
}

```

kt

实现 PAG 适配器

与字体、图片适配器的定位不同，PAG 适配器是以 工厂类 的角色向框架提供 PAGView 实例。业务可通过实现此适配器创建框架的 PAGView 组件，也可以构建自定义 PAGView，再通过 createPAGView 输出实例。

具体实现代码，请参考源码工程 androidApp 模块的 PAGViewAdapter 类。

```
class PAGViewAdapter : IKRPAGViewAdapter {

    init {
        try {
            System.loadLibrary("pag")
        } catch (e: Throwable) {

        }
    }

    override fun createPAGView(context: Context): IPAGView {
        return KRPagView(context)
    }
}

// 自定义 PAGView 示例
class KRPagView(context: Context) : PAGView(context), IPAGView {

    override fun asView(): View {
        return this
    }

    override fun setFilePath(filePath: String) {
        path = filePath
    }

    override fun playPAGView() {
        play()
    }

    override fun stopPAGView() {
        stop()
    }

    // ...
}
```

配置混淆规则（ProGuard/R8）

如果您的 Android 项目开启了代码混淆（ProGuard/R8），需要确保 Kuikly 相关的类不被混淆，以保证框架正常运行。

自动应用混淆规则

`core-render-android` 库已经通过 `consumer-rules.pro` 自动提供了必要的混淆规则，这些规则会在您引入依赖时自动应用。规则包括：

- 保留 Kuikly 核心入口类和方法
- 保留日志相关类
- 保留 RecyclerView 的反射访问方法

手动配置（可选）

如果您需要额外的混淆规则，或者想要查看库提供的规则内容，可以在您的 `app/build.gradle.kts` 或 `app/build.gradle` 中配置：

反馈文档建议

```
android {  
    buildTypes {  
        release {  
            isMinifyEnabled = true  
            proguardFiles(  
                getDefaultProguardFile("proguard-android-optimize.txt"),  
                "proguard-rules.pro"  
            )  
        }  
    }  
}
```

gradle

然后在 `app/proguard-rules.pro` 文件中添加以下规则（库已自动提供，此处仅作参考）：

```
# Kuikly 核心类保留规则  
-keep class com.tencent.kuikly.core.android.KuiklyCoreEntry { *; }  
-keep class com.tencent.kuikly.core.IKuiklyCoreEntry { *; }  
-keep class com.tencent.kuikly.core.IKuiklyCoreEntry$Delegate { *; }  
-keep class com.tencent.kuikly.core.log.KLog { *; }  
-keepnames class  
com.tencent.kuikly.core.render.android.scheduler.IKuiklyRenderCoreScheduler$* {  
    public *;  
}  
-keep class
```

proguard

```
com.tencent.kuikly.core.render.android.scheduler.KuiklyRenderCoreContextScheduler
{

com.tencent.kuikly.core.render.android.scheduler.KuiklyRenderCoreContextScheduler
INSTANCE;
    void scheduleTask(long,java.lang.Runnable);
}

# RecyclerView 反射方法保留
-keepclassmembers class androidx.recyclerview.widget.RecyclerView {
    void setScrollState(int);
}
```

配置AndroidManifest.xml

在 AndroidManifest.xml 中为您的 Activity（如 KuiklyRenderActivity ）添加以下配置：

```
<activity
    android:name=".KuiklyRenderActivity"
    android:windowSoftInputMode="stateUnspecified|adjustNothing"
    ... />
```

xml

反馈
文档
建议

配置说明

- **stateUnspecified** ：避免输入框默认获得焦点，让 Kuikly 页面可以更好地控制输入框的焦点状态
- **adjustNothing** ：避免键盘弹起时调整 Activity 的 View 大小，从而防止影响 KuiklyView 的布局和尺寸。这样可以确保 KuiklyView 的大小保持稳定，不受键盘影响

与 keyboardHeightChange 结合使用

配置 **adjustNothing** 后，Kuikly 框架可以通过 **keyboardHeightChange** 事件来监听键盘高度变化，并实现更精确的键盘规避逻辑。这种方式比系统自动调整布局更加可控，能够提供更好的用户体验。

实验性开关

Kuikly提供了一些实验性功能开关，供业务按需开启以体验新功能或优化性能。

1. `KuiklyRenderView.enableLazyClipChildren()`

- 功能描述：优化重绘范围的实验性开关，可以提升小区域动画的渲染性能，进程级开关，影响所有 Kuikly 页面，**2.16.0**版本引入
- 可能存在的缺陷：子组件超出父组件边界的部分被裁剪掉，没有正确显示

注意

实验性功能可能存在不稳定因素，务必充分测试后再在生产环境中使用。

Compose 相关配置

如果您在 Android 平台上同时使用 **Kuikly Compose** 和原生 **Jetpack Compose**，可能会出现状态丢失或 ANR 死锁问题。请参考 [Compose FAQ - enableConsumeSnapshot 配置说明](#) 进行配置。

提示：纯 Kuikly Compose 项目无需额外配置，保持默认值即可。